Estilos arquitectónicos y selección justificada para Secret Bite Web

Documento integrado de consulta decisión arquitectónica y experimento con inteligencia artificial

Sebastian Perlaza Villa

Sara Vanegas Garci

Institución: [Completar]

Asignatura: [Completar]

Docente y grupo: [Completar]

Fecha de entrega: [Completar]

Tabla de contenido

Sección	Página
Parte 1 Marco comparación fichas escenarios y selección	3
Parte 2 Prompt consulta contraste y experimento	16
Conclusiones registro de IA y ajustes pendientes	18
Anexos de la segunda parte	21

Nota de alcance

La primera parte se conserva tal como fue desarrollada por el equipo. La segunda parte contrasta esa decisión con el código real del MVP. Los campos académicos entre corchetes deben completarse antes de la entrega. Pedidos se trata como evolución futura porque no está implementado en el repositorio actual.

Estilos arquitectónicos y selección justificada — Secret Bite Web (Parte 1)

Sebastian Perlaza Villa
Sara Vanegas Garci

Para Secret Bite Web el equipo elige un **monolito modular organizado en capas, con arquitectura cliente-servidor** (SPA + API REST + base de datos). Esta sección corresponde a la primera parte de la actividad: marco de comparación, fichas de cinco estilos, cuadro comparativo, escenarios de calidad y justificación de la decisión.

Secret Bite Web es la plataforma de una hamburguesería con dos funciones: la votación mensual de la hamburguesa del mes (MVP ya generado) y la gestión de pedidos a domicilio. La decisión se documenta aquí antes de consultar al asistente de IA, como exige el paso 7 de la actividad.

1. Marco de comparación

Los estilos se comparan con ocho criterios: seis atributos de calidad del modelo de producto de ISO/IEC 25010:2011 que son relevantes para una aplicación web de pedidos y votaciones, y cuatro factores prácticos ligados a un equipo universitario pequeño y un semestre de plazo. La escala es cualitativa (Alta, Media, Baja), y cada valoración se sustenta en las fichas de la sección 2.

#	Criterio	Tipo	Qué se valora	Por qué importa en Secret Bite Web
C1	Eficiencia de desempeño	ISO/IEC 25010	Tiempo de respuesta y uso de recursos bajo carga	Picos de tráfico al abrir y cerrar la votación mensual y en horas de almuerzo y cena
C2	Escalabilidad / elasticidad	ISO/IEC 25010 (capacidad)	Facilidad de atender más usuarios agregando recursos	Crecimiento posible de clientes y sedes
C3	Fiabilidad (disponibilidad y tolerancia a fallos)	ISO/IEC 25010	Que una falla parcial no tumbe el sistema	Un pedido perdido es una venta perdida
C4	Seguridad	ISO/IEC 25010	Confidencialidad, integridad, autenticidad	Un voto por usuario; datos personales y direcciones de entrega

#	Criterio	Tipo	Qué se valora	Por qué importa en Secret Bite Web
C5	Mantenibilidad (modularidad, modificabilidad)	ISO/IEC 25010	Costo de agregar o cambiar funciones	El módulo de pedidos se agrega después del MVP de votación
C6	Facilidad de prueba	ISO/IEC 25010 (mantenibilidad)	Facilidad de aislar y probar componentes	La etapa de pruebas es parte del curso
C7	Simplicidad de construcción	Factor práctico	Curva de aprendizaje y número de piezas	Equipo de estudiantes, un semestre
C8	Costo de despliegue y operación	Factor práctico	Infraestructura, monitoreo, número de servicios	Presupuesto del proyecto ya estimado y acotado
C9	Ajuste al tamaño del equipo	Factor práctico	Si el estilo presupone varios equipos autónomos	Un solo equipo de 3 a 5 personas
C10	Facilidad de despliegue	Factor práctico	Frecuencia y riesgo de publicar cambios	Entregas incrementales durante el semestre

Los criterios C1 a C6 siguen la estructura de características de ISO/IEC 25010:2011 (ISO, 2011). Los criterios C7 a C10 retoman las características operativas con que Richards y Ford (2020) califican cada estilo (simplicidad, costo total, despliegue) y la relación entre arquitectura y organización del equipo que discuten Bass et al. (2021).

Pregunta abierta: confirmar el tamaño exacto del equipo y ajustar C9 si es necesario.

2. Fichas de estilos arquitectónicos

Se consultaron cinco estilos que organizan un sistema de formas realmente distintas: por capas técnicas (capas), por módulos de negocio en una sola unidad desplegable (monolito modular), por servicios desplegables por separado (microservicios), por eventos asíncronos (orientada a eventos) y por puertos hacia un núcleo de dominio aislado (hexagonal). Cada afirmación de las fichas cita su fuente; los diagramas están escritos en Mermaid para poder entregarlos como código fuente.

2.1 Arquitectura en capas (N-capas)

Campo	Contenido
Descripción	Organiza el sistema en capas jerárquicas; cada capa ofrece servicios a la capa superior y usa solo los de la capa inferior (Garlan y Shaw, 1994). En aplicaciones web las capas típicas son presentación (UI), lógica de negocio (BLL) y acceso a datos (DAL) (Microsoft, s.f.).
Elementos	Capas: presentación, negocio, persistencia y base de datos.

Campo	Contenido
Conectores	Llamadas a procedimiento entre capas adyacentes; la UI no debe llamar directamente a la DAL (Microsoft, s.f.).
Favorece	Separación de responsabilidades y organización del código; reemplazo de la implementación de una capa sin afectar a las demás (Garlan y Shaw, 1994; Microsoft, s.f.). Simplicidad y bajo costo, por ser el estilo más conocido (Richards y Ford, 2020).
Compromete	Las dependencias apuntan hacia abajo, así que la lógica de negocio depende del acceso a datos y probarla suele exigir una base de datos de prueba (Microsoft, s.f.). Se despliega y escala como una sola unidad: cambiar un componente obliga a probar y desplegar toda la aplicación (Microsoft, s.f.). Pasar por varias capas puede costar desempeño (Garlan y Shaw, 1994).
Cuándo conviene	Aplicaciones pequeñas o medianas, con presupuesto y tiempo ajustados, y equipos que conocen el estilo (Richards y Ford, 2020).
Cuándo no	Sistemas que necesitan alta escalabilidad, elasticidad o despliegues independientes por partes (Richards y Ford, 2020; Microsoft, s.f.).
Ejemplo real	Los protocolos de comunicación en capas (modelo ISO OSI), citados por Garlan y Shaw (1994) como el ejemplo más conocido: cada capa abstrae la comunicación a un nivel distinto, desde la conexión física hasta la aplicación, y puede cambiar sin afectar a las otras. En la web, la aplicación de referencia eShopOnWeb de Microsoft se presenta frente al modelo tradicional de N-capas para ilustrar sus límites (Microsoft, s.f.).

2.2 Monolito modular

Campo	Contenido
Descripción	Una sola unidad desplegable dividida internamente en módulos por dominio de negocio, con fronteras estrictas entre ellos. Shopify lo adoptó para lograr “más modularidad sin aumentar el número de unidades de despliegue” (Westeinde, 2019). Corresponde a la partición por dominio, frente a la partición técnica de las capas (Richards y Ford, 2020).
Elementos	Módulos de dominio (cada uno con sus propias capas internas), una base de datos compartida o con esquemas por módulo.

Campo	Contenido
Conectores	Llamadas en proceso a través de la interfaz pública de cada módulo; dependencias declaradas entre módulos (Westeinde, 2019).
Favorece	Modularidad y mantenibilidad por dominio; mantiene la simplicidad de despliegue, prueba y desempeño de un monolito, sin las llamadas de red de los microservicios (Westeinde, 2019).
Compromete	Escala como una sola unidad; una falla grave puede afectar a todo el proceso; las fronteras se degradan si no se vigilan con herramientas o revisiones (Westeinde, 2019).
Cuándo conviene	Cuando el monolito se vuelve difícil de cambiar pero los microservicios traerían más costo que beneficio; según Westeinde (2019), la arquitectura correcta depende de la etapa de crecimiento del sistema.
Cuándo no	Cuando partes del sistema necesitan escalar o desplegarse de forma independiente, o cuando varios equipos autónomos trabajan en paralelo.
Ejemplo real	Shopify, uno de los mayores códigos en Ruby on Rails, con más de mil desarrolladores durante más de una década. Su monolito era difícil de cambiar (cambios pequeños rompían pruebas no relacionadas), pero los microservicios añadían pipelines separados, infraestructura y latencia de red. Lo reorganizó en componentes con fronteras estrictas dentro del mismo despliegue (Westeinde, 2019).

2.3 Microservicios

Campo	Contenido
Descripción	Construye una aplicación como un conjunto de servicios pequeños, cada uno en su propio proceso, organizados por capacidades de negocio y desplegables de forma independiente mediante automatización (Lewis y Fowler, 2014).
Elementos	Servicios por capacidad de negocio, cada uno con sus propios datos (gestión descentralizada de datos); API gateway; infraestructura de despliegue y monitoreo.
Conectores	Llamadas remotas (HTTP/REST, RPC) o mensajería ligera: “endpoints inteligentes y tuberías tontas” (Lewis y Fowler, 2014).
Favorece	Despliegue independiente, escalabilidad por servicio, aislamiento de fallos cuando se diseña para la falla, y evolución del diseño (Lewis y Fowler, 2014; Richards y Ford, 2020).

Campo	Contenido
Compromete	Las llamadas remotas son más costosas que las llamadas en proceso; las transacciones distribuidas son difíciles y llevan a consistencia eventual; operar muchos servicios exige herramientas y monitoreo; refactorizar entre servicios y probar es más difícil (Lewis y Fowler, 2014). Es el estilo de mayor costo y menor simplicidad (Richards y Ford, 2020).
Cuándo conviene	Sistemas grandes, con varios equipos que necesitan desplegar por separado y partes con demandas de escala muy distintas.
Cuándo no	Equipos pequeños o sin experiencia en automatización y operación: el resultado depende mucho de la habilidad del equipo (Lewis y Fowler, 2014).
Ejemplo real	Netflix migró entre 2008 y 2016 de una aplicación monolítica a cientos de microservicios en la nube. La motivación fue una corrupción de base de datos en 2008 que le impidió enviar DVD durante tres días: necesitaba eliminar puntos únicos de falla escalados verticalmente y crecer con la elasticidad de la nube (Izrailevsky et al., 2016).

2.4 Arquitectura orientada a eventos

Campo	Contenido
Descripción	Los componentes no se invocan directamente: anuncian eventos y otros componentes registrados reaccionan a ellos (invocación implícita) (Garlan y Shaw, 1994). Abarca patrones distintos: notificación de eventos, transferencia de estado en el evento, event sourcing y CQRS (Fowler, 2017).
Elementos	Productores, consumidores (procesadores de eventos), canales o tópicos, y un broker o mediador de eventos (Richards y Ford, 2020).
Conectores	Publicación/suscripción asíncrona a través del broker.
Favorece	Bajo acoplamiento entre productor y consumidor; es fácil agregar consumidores sin cambiar al productor (Garlan y Shaw, 1994; Fowler, 2017). Alto rendimiento y escalabilidad en flujos de alto volumen (Kreps et al., 2011; Richards y Ford, 2020).
Compromete	El flujo lógico “no está explícito en ningún texto del programa”, lo que dificulta depurar y modificar (Fowler, 2017). El productor no sabe quién responderá ni en qué orden, y razonar sobre la corrección es difícil (Garlan y Shaw, 1994). Entregas “al menos una vez” pueden duplicar mensajes y el orden solo se garantiza por partición (Kreps et al., 2011).

Campo	Contenido
Cuándo conviene	Flujos de alto volumen, procesamiento en tiempo real o muchos consumidores independientes de los mismos hechos.
Cuándo no	Flujos transaccionales simples que requieren respuesta inmediata y consistencia fuerte, o equipos sin experiencia en sistemas asíncronos.
Ejemplo real	LinkedIn creó Kafka porque sus datos de actividad y métricas superaban el tamaño de sus datos “reales”, y necesitaba alimentar a la vez analítica fuera de línea (Hadoop) y aplicaciones en tiempo real (relevancia de búsqueda, recomendaciones). Kafka manejaba cerca de mil millones de mensajes diarios con unos 10 segundos de latencia de extremo a extremo (Kreps et al., 2011).

2.5 Arquitectura hexagonal (puertos y adaptadores)

Campo	Contenido
Descripción	Busca que la aplicación pueda ser conducida igual por usuarios, programas, pruebas automatizadas o scripts, y desarrollarse y probarse aislada de sus bases de datos y dispositivos (Cockburn, 2005). Las dependencias apuntan hacia el núcleo de dominio; Microsoft (s.f.) la agrupa con la arquitectura limpia y la de cebolla.
Elementos	Núcleo de aplicación (dominio); puertos primarios o de entrada (conducen la aplicación) y secundarios o de salida (la aplicación los conduce, p. ej., la base de datos); adaptadores específicos de cada tecnología (Cockburn, 2005).
Conectores	Interfaces (puertos) implementadas por adaptadores e inyectadas en tiempo de ejecución (Microsoft, s.f.).
Favorece	Facilidad de prueba: pruebas de regresión sin interfaz ni base de datos; reemplazo de adaptadores sin tocar la lógica de negocio (Cockburn, 2005). Netflix reportó unas 3.000 pruebas en cerca de 100 segundos (Svrtan y Makagon, 2020).
Compromete	Más interfaces, mapeos entre modelos e indirección que un diseño en capas simple; no resuelve por sí misma la escalabilidad ni el despliegue, porque organiza el interior de una aplicación (valoración del equipo con base en Cockburn, 2005, y Microsoft, s.f.).
Cuándo conviene	Cuando las fuentes de datos o integraciones externas pueden cambiar, o cuando la lógica de negocio debe probarse a fondo y de forma aislada.

Campo	Contenido
Cuándo no	Aplicaciones casi solo CRUD, con poca lógica de negocio, donde la indirección añade costo sin beneficio.
Ejemplo real	El equipo Studio Workflows de Netflix la usó porque sus datos venían de un monolito que se dividiría en microservicios en una fecha desconocida. Gracias a ella, cambió una lectura de una API JSON a una fuente GraphQL en 2 horas con un cambio de configuración de una línea, sin tocar la lógica de negocio (Svrtan y Makagon, 2020).

3. Cuadro comparativo

El monolito modular y las capas son los únicos estilos que salen bien en los cuatro factores prácticos (C7 a C10). Entre ellos, el monolito modular gana en mantenibilidad por dominio, que es el criterio decisivo para agregar el módulo de pedidos. En todas las filas, **Alta** significa “favorable para Secret Bite Web”. Cada celda se sustenta en la ficha correspondiente de la sección 2.

Criterio	Capas	Monolito modular	Microservicios	Orientada a eventos	Hexagonal
C1 Eficiencia de desempeño	Media: el paso por capas tiene costo	Alta: llamadas en proceso	Media: llamadas remotas más costosas	Alta en flujos de alto volumen	Media: algo más de indirección
C2 Escalabilidad	Baja: escala como un todo	Baja: escala como un todo	Alta: por servicio	Alta	Baja: depende del despliegue
C3 Fiabilidad	Baja: un solo proceso	Baja: un solo proceso	Alta, si se diseña para la falla	Media: desacople, pero con duplicados y orden por partición	Baja: depende del despliegue
C4 Seguridad	No diferencia: se logra con tácticas (autenticación, autorización, cifrado) en cualquier estilo	Igual	Igual, con más superficie expuesta por cada servicio en red	Igual	Igual

Criterio	Capas	Monolito modular	Microservicios	Orientada a eventos	Hexagonal
C5 Mantenibilidad	Media: partición técnica, sin fronteras de negocio	Alta: fronteras por dominio	Alta, pero refactorizar entre servicios es caro	Media: flujo implícito, difícil de seguir	Alta: adaptadores intercambiables
C6 Facilidad de prueba	Media: la lógica depende de la BD	Media	Baja: pruebas entre servicios	Baja: difícil razonar sobre el orden	Alta: núcleo probado sin BD ni UI
C7 Simplicidad	Alta	Alta	Baja	Baja	Media
C8 Costo bajo de operación	Alta: un despliegue	Alta: un despliegue	Baja: muchos servicios y monitoreo	Media: requiere broker	Alta: un despliegue
C9 Ajuste a un equipo pequeño	Alta	Alta	Baja: depende mucho del equipo	Media	Alta
C10 Facilidad de despliegue	Baja: todo se redespliega	Baja: todo se redespliega	Alta: despliegue independiente	Media	Baja: depende del despliegue

La fila C4 se valora como neutral porque la seguridad se obtiene sobre todo con tácticas que se aplican en cualquier estilo (Bass et al., 2021). La hexagonal organiza el interior de una aplicación, por eso en C2, C3 y C10 hereda el comportamiento del estilo de despliegue con que se combine.

4. Escenarios de calidad del sistema

Los siete escenarios más importantes de Secret Bite Web siguen la estructura de seis partes de Bass et al. (2021): fuente del estímulo, estímulo, artefacto, entorno, respuesta y medida de la respuesta. **Las cifras de las medidas son una propuesta: deben ajustarse a los requisitos no funcionales del ERS (IEEE 830) del proyecto antes de entregar.**

ID	Atributo	Fuente	Estímulo	Artefacto	Entorno	Respuesta	Medida
EC1	Desempeño	Clientes registrados	500 votos concurrentes en la última hora antes del cierre de la votación mensual	Módulo de votación y API	Carga pico	Registra cada voto y confirma al usuario	95 % de las solicitudes responden en 2 s o menos; 0 votos perdidos

ID	Atributo	Fuente	Estímulo	Artefacto	Entorno	Respuesta	Medida
EC2	Desempeño	Cliente	Confirma un pedido a domicilio	Módulo de pedidos	Operación normal (hora de almuerzo o cena)	Guarda el pedido y lo muestra en el panel del restaurante	Confirmación en 2 s o menos (p95); pedido visible en el panel en 10 s o menos
EC3	Disponibilidad	Falla interna	El proceso de la aplicación se cae	Servidor de aplicación	Horario de servicio (11:00 a 23:00)	Detecta la falla y reinicia el servicio automáticamente	Servicio restablecido en 5 min o menos; disponibilidad mensual de al menos 99 % en horario de servicio
EC4	Seguridad (integridad)	Usuario autenticado	Intenta votar por segunda vez en el mismo mes	Módulo de votación	Votación abierta	Rechaza el voto y registra el intento	100 % de los votos duplicados rechazados y registrados
EC5	Seguridad (confidencialidad)	Usuario no autorizado	Intenta consultar pedidos o direcciones de otro cliente	API de pedidos	Operación normal	Niega el acceso (401/403)	0 accesos indebidos en las pruebas de seguridad
EC6	Modificabilidad	Equipo de desarrollo	Agrega el módulo de pedidos después del MVP de votación	Código fuente	Tiempo de desarrollo	Integra el módulo sin cambiar el de votación	0 archivos modificados en el módulo de votación; todas sus pruebas siguen pasando
EC7	Facilidad de prueba	Probador del equipo	Ejecuta las pruebas unitarias de las	Núcleo de cada módulo	Integración continua	Ejecuta las pruebas sin base	Suite en 2 min o menos; al menos 70 % de cobertura

ID	Atributo	Fuente	Estímulo	Artefacto	Entorno	Respuesta	Medida
			reglas de negocio (un voto por mes, estados del pedido)			de datos real	de la lógica de negocio

Tensiones entre atributos. EC1 y EC3 empujan hacia estilos distribuidos (escalar y aislar fallos), mientras que la simplicidad, el costo y el tamaño del equipo (C7 a C9) empujan hacia un solo despliegue. EC6 y EC7 piden fronteras internas y una lógica aislada, que un diseño en capas puro no da. La decisión de la sección 5 resuelve esa tensión.

5. Estilo seleccionado y justificación

El equipo elige una **combinación de estilos**: un **monolito modular** como estilo principal, desplegado como **cliente-servidor** (SPA en el navegador + API REST), con **capas dentro de cada módulo** y **puertos y adaptadores** (hexagonal) solo en la frontera con la persistencia y con los servicios externos.

5.1 Justificación frente a los escenarios

Escenario	Cómo lo atiende el estilo elegido
EC1, EC2 Desempeño	Las llamadas entre módulos son en proceso, sin latencia de red (Westeinde, 2019). Si la API es sin estado, el pico de votación se atiende con varias instancias detrás de un balanceador (Microsoft, s.f.).
EC3 Disponibilidad	Monitoreo de salud y reinicio automático del proceso; mínimo dos instancias en horario de servicio. Es un compromiso aceptado (ver 5.3).
EC4, EC5 Seguridad	Tácticas transversales: autenticación, autorización por recurso y una restricción de unicidad (usuario, mes) en la base de datos (Bass et al., 2021).
EC6 Modificabilidad	Votación y Pedidos son módulos con fronteras explícitas: agregar Pedidos no toca Votación, como en Shopify (Westeinde, 2019).
EC7 Facilidad de prueba	Los repositorios son puertos con adaptadores intercambiables: las reglas de negocio se prueban con adaptadores en memoria, sin base de datos (Cockburn, 2005; Microsoft, s.f.).

5.2 Alternativas descartadas

Alternativa	Por qué se descarta
Capas puras	Su partición es técnica: votación y pedidos quedarían mezclados en cada capa, lo que debilita EC6. Además, la lógica depende de la base de datos, lo que debilita EC7 (Microsoft, s.f.).
Microservicios	Sus ventajas (escala por servicio, despliegue independiente) no compensan su costo para un equipo pequeño en un semestre: operación, monitoreo, consistencia eventual y pruebas entre servicios (Lewis y Fowler, 2014; Richards y Ford, 2020). Los picos de EC1 se atienden escalando el monolito completo.
Orientada a eventos como estilo principal	El volumen de Secret Bite Web no se acerca al que justificó este estilo en LinkedIn (Kreps et al., 2011). Su flujo implícito complicaría la depuración y las pruebas (Fowler, 2017).
Hexagonal en todo el sistema	Su indirección no aporta en las partes que son casi solo CRUD (menú, perfil). Se aplica solo donde hay cambio esperado: persistencia e integraciones.

5.3 Compromisos que el equipo acepta

- **Escala como un todo**: si solo la votación tiene picos, igual se replica todo el sistema (Microsoft, s.f.).
- **Un solo proceso**: una falla grave afecta a la votación y a los pedidos a la vez. Se mitiga con reinicio automático y dos instancias.
- **Despliegue único**: un cambio en Pedidos obliga a redesplegar todo y a correr todas las pruebas.
- **Disciplina de fronteras**: los módulos solo se comunican por su interfaz pública; se verifica en revisión de código, porque las fronteras se degradan si no se vigilan (Westeinde, 2019).
- **Ruta de evolución**: si la votación creciera mucho, su módulo podría extraerse como servicio, porque ya tiene fronteras y datos propios.

5.4 Diagrama de la arquitectura

```
flowchart TB
  U["Cliente web (SPA en el navegador)"] -- "HTTPS / JSON" --> LB["Balanceador de
carga"]
  subgraph API["API REST – monolito modular (N instancias sin estado)"]
    direction LR
    subgraph AUTH["Módulo Usuarios y autenticación"]
      A1["Controlador"] --> A2["Servicio"] --> A3["Puerto repositorio"]
    end
    subgraph VOT["Módulo Votación"]
```

```
    V1["Controlador"] --> V2["Servicio"] --> V3["Puerto repositorio"]
  end
  subgraph MEN["Módulo Menú"]
    M1["Controlador"] --> M2["Servicio"] --> M3["Puerto repositorio"]
  end
  subgraph PED["Módulo Pedidos"]
    P1["Controlador"] --> P2["Servicio"] --> P3["Puerto repositorio"]
  end
  V2 -. "interfaz pública" .-> M2
  P2 -. "interfaz pública" .-> M2
  V2 -. "interfaz pública" .-> A2
  P2 -. "interfaz pública" .-> A2
  ADP["Adaptadores SQL"]
  A3 --> ADP
  V3 --> ADP
  M3 --> ADP
  P3 --> ADP
end
LB --> API
ADP --> DB[("Base de datos relacional<br/>un esquema por módulo")]
```

Figura 6. Arquitectura de Secret Bite Web: monolito modular con capas por módulo y puertos hacia la persistencia.

Las flechas punteadas son las únicas dependencias permitidas entre módulos, siempre a través de la interfaz pública del servicio. Pendiente de confirmar con el equipo: los nombres de los módulos y la tecnología de base de datos del MVP.

Material de consulta

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture* (HaT Technical Report 2005.02). https://alistair.cockburn.us/hexagonal-architecture/

Fowler, M. (2017, 7 de febrero). *What do you mean by "event-driven"?* martinfowler.com. https://martinfowler.com/articles/201701-event-driven.html

Garlan, D., & Shaw, M. (1994). *An introduction to software architecture* (CMU-CS-94-166 / CMU/SEI-94-TR-21). Carnegie Mellon University. https://ocw.mit.edu/courses/16-355j-software-engineering-concepts-fall-2005/4b3577da7baf3801621e092a5fa32b92_intro_softarch.pdf

International Organization for Standardization. (2011). *ISO/IEC 25010:2011 — Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models*. ISO.

Izrailevsky, Y., Vlaovic, S., & Meshenberg, R. (2016, 12 de febrero). *Completing the Netflix cloud migration*. Netflix. https://about.netflix.com/en/news/completing-the-netflix-cloud-migration

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the NetDB'11 Workshop*. Atenas, Grecia. https://people.csail.mit.edu/matei/courses/2015/6.S897/readings/kafka.pdf

Lewis, J., & Fowler, M. (2014, 25 de marzo). *Microservices: A definition of this new architectural term*. martinfowler.com. https://martinfowler.com/articles/microservices.html

Microsoft. (s.f.). *Common web application architectures*. En *Architect modern web applications with ASP.NET Core and Azure*. Microsoft Learn. Recuperado el 19 de octubre de 2026, de https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Svrtan, D., & Makagon, S. (2020, 10 de marzo). *Ready for changes with hexagonal architecture*. Netflix Technology Blog. https://netflixtechblog.com/ready-for-changes-with-hexagonal-architecture-b315ec967749

Westeinde, K. (2019, 21 de febrero). *Deconstructing the monolith: Designing software that maximizes developer productivity*. Shopify Engineering. https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity

Nota de verificación (para el registro de uso de IA). Las fuentes web se abrieron y contrastaron el 19 de octubre de 2026. Los libros de Bass et al. (2021) y Richards y Ford (2020), y la norma ISO/IEC 25010, se citan por su contenido general (escenarios de seis partes, tácticas, calificaciones de estilos). Antes de entregar, el equipo debe ubicar en ellos el capítulo o la página de cada afirmación.

6 Prompt original y contexto de generación

El código se generó en Codex para escritorio con agentes de arquitectura y desarrollo. La familia del modelo fue GPT-5; la versión exacta no quedó visible en el registro exportado. El prompt inicial indicó las tecnologías, pero no nombró un estilo arquitectónico ni pidió compararlo antes de programar.

Prompt inicial conservado

```
Necesito realizar una aplicacion con javaScript en express y con react hooks donde podamos desarrollar el README que t
```

El pedido de React y Express determinó indirectamente una relación cliente-servidor. En cambio, la organización interna del backend quedó abierta. El agente arquitecto recomendó un monolito modular con capas y repositorios; el código generado conservó un monolito simple con separación técnica básica.

7 Consulta independiente sobre el código actual

La consulta se realizó en un contexto separado y se contrastó con el repositorio. La respuesta identifica el estilo implementado y distingue la arquitectura actual de la arquitectura objetivo.

Pregunta

```
¿Qué estilo arquitectónico emplea este código, cómo se manifiesta y cuál debió emplearse frente a los escenarios de ca
```

Respuesta documentada

El código emplea una arquitectura cliente-servidor: una SPA React consume una API REST Express. El backend es un monolito con una base SQLite compartida. Existe una separación técnica básica entre presentación, API y persistencia, pero no un monolito modular formal. Autenticación, hamburguesas, campañas y votos no están aislados mediante módulos con interfaces públicas; además, varios handlers HTTP ejecutan SQL directamente. Por ello tampoco hay una arquitectura hexagonal completa.

Componente	Evidencia en el repositorio	Interpretación
Cliente	client/src/App.jsx y client/src/api.js	SPA React que consume JSON por HTTP
Servidor	server/src/app.js	API REST Express; rutas, validación y reglas concentradas
Persistencia	server/src/db.js y SQLite	Esquema relacional; repositorio parcial
Despliegue	Un proceso Node y una base local	Monolito desplegado como unidad
Pruebas	server/test/api.test.js	Integración con SQLite en memoria, no puertos simulados

Tabla 1. Evidencia del estilo implementado en el repositorio.

Para el MVP actual, el monolito simple mantiene bajo el costo de construcción. Para incorporar pedidos y otras capacidades sin degradar la mantenibilidad, el sistema debería evolucionar hacia un monolito modular cliente-servidor. Cada dominio tendría controller, service y repository propios, y la persistencia se aislaría mediante puertos y adaptadores. Microservicios y eventos añadirían complejidad operativa desproporcionada para el alcance y el equipo.

8 Contraste de decisiones arquitectónicas

Aspecto	Estilo empleado	Estilo recomendado por la consulta	Decisión previa del equipo
Estilo principal	Monolito simple con capas técnicas	Monolito modular	Monolito modular
Distribución	SPA React + API REST + SQLite	Cliente-servidor	Cliente-servidor
Interior	Rutas y reglas juntas en app.js; SQL también en handlers	Controller-Service-Repository por dominio	Capas por módulo y puertos selectivos
Pruebas	Integración con SQLite en memoria	Reglas con repositorios falsos más integración real	Núcleo aislado de la base
Clasificación	Cliente-servidor indicado indirectamente; monolito simple espontáneo	Propuesto después de revisar código y escenarios	Decisión documentada antes de la consulta

Tabla 2. Contraste entre el código, la consulta y la decisión del equipo.

La diferencia es verificable: el diseño del equipo y la recomendación del asistente coinciden en el objetivo modular, pero el código no materializa todavía esas fronteras. Pedir tecnologías no fue suficiente para obtener la estructura interna esperada.

9 Experimento de reformulación

Se ejecutaron dos variantes aisladas con el mismo tipo de asistente y el mismo alcance: un esqueleto del módulo Votación. La primera pidió estudiar alternativas; la segunda impuso el estilo y reglas de dependencia.

Variante A estudio previo

```
Antes de generar código, compara capas, monolito modular, microservicios y hexagonal frente a 500 votos concurrentes, p
```

Resultado. El asistente eligió un monolito modular con arquitectura hexagonal dentro de Votación. Propuso módulos de negocio, un puerto VoteRepository, un caso de uso CastVote, adaptador PostgreSQL, pruebas con un repositorio falso y una prueba de carga. También aclaró que p95 y cero votos perdidos deben medirse; la arquitectura por sí sola no los demuestra.

Variante B estilo explícito

```
Genera el módulo Votación de Secret Bite con un monolito modular cliente-servidor. Dentro del módulo usa Controller ->
```

Resultado. El asistente siguió la estructura pedida sin volver a debatir el estilo. Produjo una API pública del módulo, carpetas http, application, ports, infrastructure y domain, y separó adaptadores PostgreSQL y SQLite. La salida fue más determinista y más fácil de revisar contra reglas de dependencias.

Criterio	Original	Variante A estudia	Variante B explícita
Elección	Monolito simple espontáneo	Elige modular + hexagonal	Aplica modular + capas + puerto
Fronteras	No verificables	Propone módulos por negocio	Nombra carpetas y dependencias permitidas
SQL en handlers	Sí	Lo evita mediante repositorio	Lo prohíbe expresamente
Pruebas sin DB	No; usa SQLite en memoria	FakeVoteRepository	InMemoryVoteRepository
Riesgo	Desviación respecto del diseño	Puede elegir otra opción si cambian supuestos	Puede obedecer un estilo inadecuado si el equipo se equivoca

Tabla 3. Comparación de las tres generaciones.

10 Conclusiones y recomendación

- El asistente eligió espontáneamente un monolito simple porque el prompt original fijó tecnologías y alcance, pero no fronteras internas.
- Cuando se le pidió estudiar alternativas, eligió una combinación coherente con la decisión previa del equipo y explicó los compromisos.
- Cuando el estilo se indicó de forma explícita, la estructura fue más consistente y verificable: ningún controller ejecutó SQL y las reglas se probaron mediante puertos.
- La mejor estrategia combina ambos enfoques: pedir primero una comparación breve basada en escenarios y, después de aprobar la decisión, generar con reglas arquitectónicas explícitas y criterios de aceptación.

Recomendación de prompt

```
Analiza primero capas, monolito modular, microservicios y hexagonal frente a los escenarios de calidad adjuntos. Prese
```

Limitaciones del experimento

- Se evaluó un solo sistema y una sola familia de modelo.
- Las respuestas de un modelo generativo pueden variar entre ejecuciones.

- Las variantes produjeron esqueletos, no una segunda aplicación completa.
- El objetivo de 500 votos y p95 menor de dos segundos no se verificó con una prueba de carga en infraestructura productiva.
- Pedidos sigue siendo una capacidad futura; no está implementada en el MVP revisado.

11 Registro de uso de inteligencia artificial

Herramienta	Uso	Versión registrada	Verificación humana
Codex escritorio	Inspección del repositorio y desarrollo del MVP	Familia GPT-5; versión exacta no visible	Se ejecutaron build, seed y cuatro pruebas
Agente documentador	Comparó guía, PDF y código	Misma plataforma; versión exacta no visible	Se contrastaron rutas, esquema y pruebas
Dos ejecuciones aisladas	Variantes A y B del experimento	Mismo tipo de agente	Se compararon estructura, dependencias y limitaciones

Prompt	Cambio principal	Motivo
Original	Solo tecnologías y alcance	Construir el MVP
Consulta	Entrega el código y pregunta estilo usado/debido	Separar evidencia del código de la decisión previa
Variante A	Obliga a comparar antes de generar	Evaluar si la IA razona la elección
Variante B	Fija estilo y reglas verificables	Evaluar obediencia y consistencia estructural
Mejorado	Dos etapas: decisión aprobada y generación restringida	Reducir decisiones implícitas y facilitar revisión

Tabla 4. Evolución de los prompts principales.

12 Ajustes pendientes antes de entregar

- Completar institución, asignatura, docente, grupo y fecha en la portada.
- Confirmar los nombres y tildes de todos los integrantes y el tamaño real del equipo.
- Decidir si Pedidos pertenece al alcance evaluado o se presenta únicamente como evolución futura.
- Ajustar las métricas de los escenarios al ERS aprobado y no afirmar alta disponibilidad ni balanceo como funciones implementadas.
- Renderizar los diagramas de cada estilo en la primera parte; conservar su código Mermaid en anexos.

- Agregar capítulo o página de libros y norma ISO cuando el equipo tenga acceso a los ejemplares consultados.

Referencias complementarias

- Cockburn, A. (2005). Hexagonal architecture. https://alistair.cockburn.us/hexagonal-architecture/
- Fowler, M. (2017, 7 de febrero). What do you mean by event-driven? https://martinfowler.com/articles/201701-event-driven.html
- Lewis, J., & Fowler, M. (2014, 25 de marzo). Microservices. https://martinfowler.com/articles/microservices.html
- Microsoft. (s. f.). Common web application architectures. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures
- Svrtan, D., & Makagon, S. (2020, 10 de marzo). Ready for changes with hexagonal architecture. Netflix Technology Blog.
- Westeinde, K. (2019, 21 de febrero). Deconstructing the monolith. Shopify Engineering.

Anexos de la segunda parte

Transcripciones y fragmentos relevantes del experimento

Anexo A Instrucción operacional usada para generar el MVP

```
Implementa el MVP completo siguiendo la propuesta del agente arquitecto. Crea un proyecto ejecutable. Alcance: React H
```

Anexo B Evidencia del código original

```
client/src/App.jsx          SPA, autenticación y panel
client/src/api.js           cliente HTTP
server/src/app.js           rutas, JWT, roles, validación y SQL
server/src/db.js            esquema SQLite y consultas de campañas
server/test/api.test.js     concurrencia, cierre, roles y sesión
```

Restricción de integridad implementada

```
UNIQUE(user_id, campaign_id)
BEGIN IMMEDIATE;
INSERT INTO votes(user_id, campaign_id, burger_id) VALUES (?, ?, ?);
COMMIT;
```

Anexo C Fragmento representativo de la variante A

```
modules/voting/
  domain/vote.js
  application/cast-vote.js
  application/vote-repository.js
  infrastructure/http/vote-router.js
  infrastructure/persistence/pg-vote-repository.js
```

```
class CastVote {
  constructor({ votes, campaigns, ids, clock }) {
    Object.assign(this, { votes, campaigns, ids, clock });
  }
  async execute(input) {
    const campaign = await this.campaigns.findOpenWithOption(input.campaignId, input.optionId);
    if (!campaign) throw new CampaignUnavailableError();
    const created = await this.votes.create({ ...input, id: this.ids.next(), createdAt: this.clock.now() });
    if (!created) throw new DuplicateVoteError();
    return created;
  }
}
```

Anexo D Fragmento representativo de la variante B

```
modules/voting/
  index.js
  http/VoteController.js
  application/VoteService.js
  ports/VoteRepository.js
  infrastructure/PostgresVoteRepository.js
  infrastructure/SqliteVoteRepository.js
  domain/errors.js
```

```
async createUnique(vote) {
  const { rows } = await this.pool.query(
    `INSERT INTO votes (user_id, campaign_id, option_id, created_at)
     VALUES ($1,$2,$3,$4)
     ON CONFLICT (user_id, campaign_id) DO NOTHING
     RETURNING *`,
    [vote.userId, vote.campaignId, vote.optionId, vote.createdAt]
  );
  return rows.length ? { created: true, vote: rows[0] } : { created: false };
}
```

Anexo E Código Mermaid de la arquitectura objetivo

```
flowchart TB
  SPA[React SPA] -->|HTTPS JSON| API[Express API]
  subgraph MONO[Monolito modular]
    AUTH[Usuarios]
    CAT[Catalogo]
    CAMP[Campanas]
    VOTE[Votacion]
    VOTE --> VC[Controller]
    VC --> VS[Service]
    VS --> VR[VoteRepository port]
  end
  API --> AUTH
  API --> CAT
  API --> CAMP
  API --> VOTE
  VR --> PG[(PostgreSQL)]
  VR -. desarrollo .-> SQ[(SQLite)]
```

Anexo F Resultados verificables del MVP

- TC-02: diez solicitudes concurrentes del mismo cliente dejan un único voto.
- TC-01: un voto emitido después del cierre es rechazado.
- TC-03: un usuario CLIENT no puede crear campañas y recibe 403.
- Registro, login y consulta del usuario autenticado funcionan en la prueba de integración.
- La compilación Vite terminó correctamente y el endpoint /api/health respondió ok.